Rafiq — Employee AI Companion

Muhammad Omaid — Magnus Mage (dev team) · entry for the Ignyte × Emirates **Employee AI Companion** challenge · prototype + this note · all data synthetic

An employee-owned companion that carries a person through **both** required journeys — onboarding and internal mobility — knows where they are against service-level targets, answers policy questions with inline citations, starts and routes routine requests, and catches friction **before** it becomes a complaint — on a consent-first, in-region, UAE-sovereign footing.

1 · Problem and scope

At Emirates scale (131,000+ employees, ~9,700 UAE joiners last year) one employee's experience spans many teams, systems, approvals and policies. The warning signals exist — a journey event that never happened, a request sitting past its SLA, a dip in an opt-in pulse — but they are scattered across systems and surface only **after** a complaint or escalation. Employees don't know where to go; HR finds out late; service owners receive incomplete requests.
Scope: two journeys only (onboarding, internal mobility). **Out of scope:** the full employee lifecycle, connection to live Emirates systems, any use in performance or disciplinary decisions — enforced by design throughout.

2 · What we built — four deterministic-first parts and a trust layer

Part	What it does	Where
Journey-state engine	Deterministic rule join of journey events against journey_maps.md target days — day past target with no completion ⇒ blocker with the map's named owner. Any hidden stall scenario falls out of this one rule; no scenario-specific code.	app/core/journey_engine.py
Grounded answer layer	BM25 retrieval with Arabic-aware normalization + heading anchors over 17 bilingual policies (EN/AR); every answer cites policy_id · section; below the floor: “I'm not sure” + handoff — enforced by one trust_gate wrapper, not prompting discipline.	app/core/rag.py, guardrails.py
Action & routing	Rules-first orchestrator: sensitive → human, status → journey, request verbs → action, else policy. Chases and requests produce structured handoff cards routed to named owners.	app/agents/orchestrator.py, action_agent.py
Friction radar + sweep	Explainable weights: +0.25 overdue event, +0.15 past-SLA request, +0.10 consented pulse dip. The employee always sees the blocker first; only persistent stalls (≥2 days) escalate. Idempotent per employee/event/day.	app/core/friction.py, services/scheduler.py
Trust layer	Consent store, employee-visible append-only audit log, PII redaction on every reply, sensitive-topic guard, one trust_gate wrapping every answer.	app/core/consent.py, guardrails.py

LLM calls live in exactly one file (app/core/llm.py): without a key, answers are extractive from retrieved policy (same citations, zero hallucination surface); with a key, temperature-0 generation strictly inside the retrieved context. The sovereign path — Falcon-H1 Arabic or JAIS on-premise, in-region — is a one-file swap.

3 · Consent, privacy and human oversight

- **Opt-in consent, enforced at the signal level.** Pulse feedback contributes to friction only while ConsentStore records opt-in; revocation excludes the signal immediately and is confirmed to the employee. Answering a pulse without consent is acknowledged but explicitly not used (test_consent_gates_pulse_signal).
- **The employee sees their own data.** A “what we hold about you” panel (profile, line manager, HRBP, journey state, requests, pulse count) plus an append-only audit log of every action the companion took — employee-visible at all times.
- **Human handoffs, named owners.** Every intervention carries a named human owner. Sensitive topics (grievance, medical circumstances, visa problems) route to a person before any answer is attempted. Owners work a complete inbox (stage, blocker, what's been tried); their acknowledge/resolve actions appear in the employee's audit log.
- **Not a monitoring tool.** The manager view structurally contains no pulse data — the endpoint never reads it (test_team_view_groups_reports). HRBP and analytics views are aggregate only, never individual. Scores never feed performance or discipline.
- **Data stays home.** Fully self-hosted (FastAPI + pandas + dependency-free BM25); the pack is never uploaded to services that retain or train on data; in-region inference keeps personal data inside the UAE (PDPL-aligned).

4 · Requirements coverage

Brief requirement	How Rafiq meets it
Knows the journey stage and what's next (day 5, laptop not issued)	Deterministic engine + timeline + proactive check-ins; the Home screen opens on the live blocker
Answers policy questions, citing the source policy	Grounded answer cards with policy_id · section chips; low confidence ⇒ “I'm not sure” + handoff
Completes/starts routine requests, handing to HR, IT, learning workflows	Request routing to Payroll, IT Service Desk, Learning & Development, Facilities, HR letters — with journey context attached
Spots friction early (events, request delays, opt-in pulse)	Friction radar + scheduled sweep; employee sees it first, then the named owner
Escalates to the right owner with a summary	Handoff inbox: complete cards with acknowledge/resolve, mirrored in the employee's audit log
Arabic + English · manager view · where journeys stall	True RTL flip + Arabic corpus/answers; risk-sorted team view (pulse-free); aggregate stall map by stage and owner

5 · Pilot readiness and deployability

- **39 tests green** across the engine, friction weights, RAG grounding, routing, sweep and trust surfaces — the guardrails are asserted, not promised; CI runs them on every push.
- **Configuration, not code:** confidence floor, escalate-after threshold, sweep interval and the demo clock all live in .env; thresholds are tuned against the pack, not by feel.
- **Pack-ready by design:** the official synthetic pack is published only at launch, so Rafiq is built to the documented schema — drop the official files into data/pack/ and the alias loader ingests them, and the judges' hidden scenarios, with zero code change. The policy-agnostic corpus is packaged for reuse by other government entities.

6 · Reproduce it

The prototype runs from the repository in three commands — no API keys, no cloud services, no data leaves the machine:

`uv sync` · `uv run uvicorn app.main:app --port 8000` · `uv run pytest -q` (39 tests)

Demo walkthrough: `make run` then follow the two-minute tour in the repository README; the 1:46 recorded demo with narration and captions ships alongside this note, and the full pipeline that produced it is re-runnable from submission/video/make_video.py.

7 · Limitations and next steps

- Pack stays read-only in the prototype (writes go to runtime stores); a pilot writes back through read-only-first connectors.
- BM25 over 17 policies is deliberately dependency-free; hybrid search + cross-encoder reranker before the corpus passes ~50 documents.
- Rules-first intent routing (auditable today); a small learned router once real phrasings exist. Voice designed, not built — text keeps answers cited and auditable.